SOHelioraX ��� README

SOHelioraX — SolarShield AI

AI-Powered Solar Flare Nowcasting & Risk Forecasting System

*Built on real ISRO **Aditya-L1** data (SoLEXS + HEL1OS)*

📑 Table of Contents

- Overview
- The Problem
- Our Solution
- Key Features
- Tech Stack
- System Architecture
- Dataset
- Machine Learning Model
- Installation & Setup
- Running the Project
- Project Structure
- Deployment
- Honest Limitations
- Team
- Acknowledgements

🌞 Overview

SOHelioraX is an end-to-end space-weather intelligence platform that detects, classifies, and forecasts solar flares in real time using data from India's first dedicated solar mission, **Aditya-L1**. It fuses two on-board instruments — **SoLEXS** (Soft X-ray Spectrometer) and **HEL1OS** (High Energy L1 Orbiting X-ray Spectrometer) — with a trained machine-learning model to produce flare classifications (A/B/C/M/X), a 4-level operational risk score, short-term probability forecasts, and early-warning alerts, all served through an interactive web dashboard.

! The Problem

- Solar flares release intense X-ray and energetic-particle radiation that can disrupt **satellites, GPS, aviation, power grids, and communication systems**.
- Most operational space-weather tools rely on **NOAA / GOES** data and **fixed thresholds**, not on India's own Aditya-L1 mission.
- Aditya-L1 Level-1 data is largely used for **post-event scientific study** — there is no easy, real-time, human-readable system that turns this raw telemetry into **actionable warnings**.

✔ Our Solution

SOHelioraX turns raw Aditya-L1 telemetry into instant, human-readable intelligence:

1. **Reads** live SoLEXS & HEL1OS Level-1 FITS files.
2. **Extracts** statistical X-ray light-curve features.
3. **Classifies** the flare (A–X) using a Random Forest model + physics blend.
4. **Scores** a 4-level operational risk (Green / Yellow / Orange / Red).
5. **Forecasts** 1H / 2H / 4H flare probability with confidence scores.

6. **Alerts** subscribers via email / SMS with an estimated lead time.

🚀 Key Features

Feature	Description
🛰️ **Live Data Pipeline**	Reads live SoLEXS & HEL1OS Level-1 FITS files from the ISSDC PRADAN portal with automatic feature extraction.
🧠 **Nowcasting Engine**	Physics + ML hybrid — Random Forest (300 trees) classifies real-time flares (A–X) at **97.7% ± 1.1%** accuracy.
📈 **Forecasting Panel**	1H / 2H / 4H probabilistic forecasts with dynamic confidence scores.
🛡️ **Dual Risk Gauges**	Live risk gauge + a 739-day historical risk baseline (Green → Red, 4-level).
⏱️ **Lead-Time Alerts**	~30-minute estimated early warning before flare peak, with email/SMS notifications.
🗓️ **Interactive Explorer**	Browse all **739 observation days** on a month-wise calendar grid with per-day flare analytics.
📊 **Live Visualizations**	Real-time SoLEXS light curves, 739-day activity trend, and flare-class distribution.

🧰 Tech Stack

Backend & ML

- Python 3.10+
- Flask (web framework)
- scikit-learn (Random Forest classifier + regressor)
- pandas, NumPy (data processing)
- Astropy (reading FITS files)
- joblib (model persistence)
- Matplotlib (server-side graphs)

Frontend

- HTML5, CSS3, JavaScript (dark, dashboard-style UI)
- Custom SVG gauges & responsive panels

Deployment

- Gunicorn (WSGI server)
- Render (cloud hosting)

🏗️ System Architecture



Flow: Raw Space Data → Process & Prepare → AI Analysis & Prediction → Dashboard & Visualization → Alerts & Insights

🛰️ Dataset

- **Source:** ISRO Aditya-L1 mission — **SoLEXS** (Soft X-ray) & **HEL1OS** (Hard X-ray) Level-1 data, downloaded from the **ISSDC PRADAN** portal.
- **Coverage: 739 days** of real observational data.
- **Features extracted per day (8 total):**
  - SoLEXS: `peak`, `mean`, `std`, `median`, `p95`
  - HEL1OS: `peak`, `mean`, `std`
- **Labeling:** Flare class is assigned from the **SoLEXS soft X-ray peak** (GOES-standard) using realistic logarithmic thresholds — missing/zero-data days are skipped.

Flare classification thresholds (SoLEXS soft X-ray peak counts):

Class	Threshold	Meaning
A	< 150	Quiet / very low
B	< 750	Minor
C	< 3,500	Moderate
M	< 18,000	Strong
X	≥ 18,000	Extreme (rare)

🤖 Machine Learning Model

- **Algorithm:** Random Forest Classifier (`n_estimators=300`, `class_weight="balanced"`).
- **Validation:** Honest **5-fold Stratified Cross-Validation**.
- **Accuracy: 97.7% ± 1.1%** (cross-validated mean ± std).
- **Outputs:** flare class (A/B/C/M/X), C/M/X probabilities, confidence score, and a 0–100 risk score.
- **Hybrid logic:** ML class probabilities are blended 50/50 with physics-based (peak-derived) probabilities for robustness.
- **Secondary model:** a Random Forest Regressor estimates a continuous peak-based risk score.

Risk levels (aligned with flare classes):

Risk	Class	Peak Range
🟢 Green	A / B	< 750
🟡 Yellow	C	750 – 3,500
🟠 Orange	M	3,500 – 18,000
🔴 Red	X	18,000+

⚙️ Installation & Setup

Prerequisites

- Python 3.10 or higher
- pip

Steps

```
# 1. Clone the repository
git clone https://github.com/<your-username>/SOHelioraX.git
cd SOHelioraX/Backend

# 2. (Recommended) create a virtual environment
python -m venv .venv
.venv\Scripts\activate        # Windows
# source .venv/bin/activate   # macOS/Linux

# 3. Install dependencies
```

```
pip install -r requirements.txt
```

requirements.txt

```
flask
numpy
pandas
scikit-learn
joblib
astropy
matplotlib
gunicorn
```

▶ Running the Project

> ⚠ Always run the Flask app from the **integrated terminal** (`python app.py`), not the "Run" (Code Runner) button.

Full pipeline (first time / after new data):

```
python build_dataset.py     # build training dataset (CSV) from Aditya-L1 data
python train_model.py       # train the Random Forest model
python model_stats.py       # generate model stats (accuracy, features)
python explorer_data.py     # generate the 739-day explorer data
python make_sample_data.py  # create sample data for deployment
```

Run the web app:

```
python app.py
```

Then open **http://127.0.0.1:5000** in your browser.

> 💡 The app is self-healing: if live SoLEXS files aren't found, it automatically falls back to committed sample data so the dashboard never crashes.

🗂 Project Structure

```
Backend/
├── app.py                  # Flask backend (routes, ML integration, alerts)
├── ml_predictor.py         # Prediction logic (risk + class + confidence)
├── build_dataset.py        # Builds training dataset from Aditya-L1 FITS
├── train_model.py          # Trains Random Forest (with cross-validation)
├── model_stats.py          # Generates model accuracy/stats JSON
├── explorer_data.py        # Generates 739-day explorer JSON
├── make_sample_data.py     # Creates lightweight sample data for deploy
├── graph_generator.py      # Live SoLEXS light-curve graph
├── templates/
│   ├── dashboard.html      # Main dashboard UI
│   └── explorer.html       # Data explorer UI
├── static/                 # Generated graphs, assets, favicon
├── ml_data/                # Dataset CSV, explorer JSON, sample data
├── ml_models/              # Trained .pkl models + model_stats.json
└── requirements.txt
```

☁ Deployment

Deployed on **Render** as a web service:

1. Push the repository to GitHub.
2. On render.com → **New → Web Service** → connect the repo.
3. **Root Directory:** `Backend`

4. **Build Command:** `pip install -r requirements.txt`
5. **Start Command:** `gunicorn app:app`
6. Deploy → get a live URL.

The committed `ml_data/` (CSV, JSON, sample data) and `ml_models/` (.pkl) let the app run on the server without needing the raw Aditya-L1 archive.

⚖ Honest Limitations

We believe in transparent engineering:

- **Live panel:** the deployed version shows committed **sample SoLEXS data** (not a real-time feed), since a live Aditya-L1 stream isn't publicly available. All other components (ML model, forecasts, explorer, accuracy) run on real data.
- **Lead time (~30 min):** an **estimate** based on flare rise-time heuristics, not a validated forecasting metric.
- **Email alerts:** fully functional with a free Gmail app-password (via environment variables).
- **SMS alerts:** the phone number is saved and the pipeline is ready, but sending real SMS requires a paid gateway (e.g., Twilio / Fast2SMS).
- **Flare class** is peak-defined per the GOES standard; the model classifies from the full statistical profile of SoLEXS + HEL1OS.

👥 Team

Bharatiya Antariksh Hackathon 2026 Sharda University, Greater Noida

Role	Name
Team Leader	Archita Tripathi
Team Member	Manthan Dhanraj

🙏 Acknowledgements

- **ISRO** & the **Aditya-L1** mission team for the SoLEXS & HEL1OS data.
- **ISSDC PRADAN** portal for data access.
- Open-source libraries: Flask, scikit-learn, Astropy, pandas, NumPy, Matplotlib.

SOHelioraX — Real-time Space Weather Intelligence ⚡

Accurate Nowcasting · Early Warnings · Operational Readiness